



PUC Minas

Design Patterns

Messias Bittencourt

Design Patterns

- Design Patterns significa Padrões de Projeto que visam resolver problemas comuns de forma padronizada no mundo.
- A vantagem de usar Design Patterns é que são amplamente conhecidos e independente do seu produto, ao se falar de um Design Patterns, sua implementação tende a facilitar o dia a dia de quem está implementando código.
- Na Unidade 3 veremos como implementar alguns desses Design Patterns que melhorarão nossa arquitetura como um todo.

Facade

- Facade é um Design Pattern que tem como objetivo criar uma interface simples pra integrar com subsistemas complexos.

Exemplo:

- Se temos diversas ações que são obrigatoriamente feitas para um objetivo, elas podem ser agrupadas de maneira a liberar uma interface simples pra quem precisar fazer uso do recurso final.

Decorator

- Decorator é um Design Pattern que serve pra adicionar funcionalidades a uma determinada ação sem modificar a ação propriamente dita.

Exemplo:

- Uma determinada função vai ser executada mas eu preciso sempre adicionar uma ação antes e/ou depois dela, como um Log por exemplo, o Decorator é um excelente Design Pattern pra isso.

Singleton

- Singleton é usado para quando precisamos ter apenas uma única instância de uma determinada classe.

Exemplo:

- Uma classe que faz a conexão com o banco de dados, talvez seja interessante para o projeto que apenas uma instancia dela esteja ativa pra evitar que multiplas conexões sejam feitas desnecessariamente.

Adapter

- O Adapter é usado quando precisamos que diferentes interfaces trabalhem juntas sem precisar modificar a interface interna.
- Exemplo:
- Um sistema que funciona de forma que ele recebe um XML como parâmetro de entrada pode-se precisar de um Adapter pra aceitar JSON como parâmetro de entrada sem que isso modifique sua forma de funcionamento.



PUC Minas